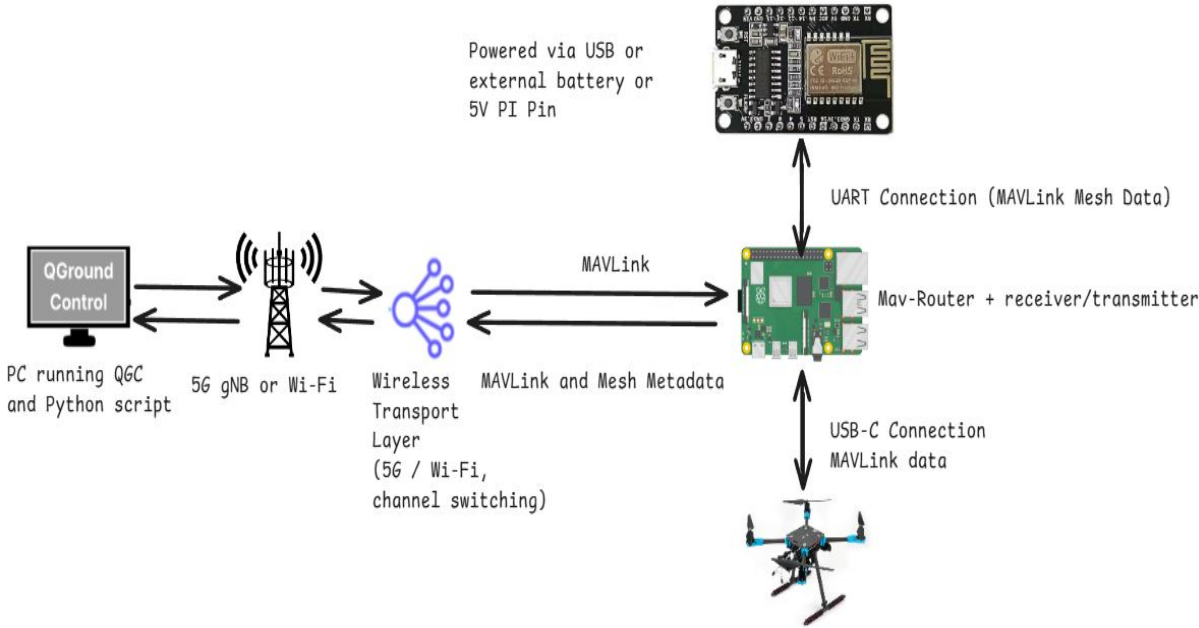
Drone Garage Updates

11/14/2025

System Architecture

This architecture forwards MAVLink telemetry between QGroundControl (QGC) and the drone over an IP based wireless link (5G or Wi-Fi). A Raspberry P runs MAVLink Router to relay data between the flight controller and an ESP-based mesh node.

The ESP communicates with the Pi over UART and provides additional mesh metadata, such as which node is leader and network health information. The Pi merges this metadata with MAVLink or forwards it separately to the ground station



The wireless transport layer is abstract, meaning the system can use either 5G or Wi-Fi, and can switch between them or different channels without changing the MAVLink or mesh logic

Mesh Connection

- Leader election works (https://github.com/Dcatna/AeroShieldSD/blob/main/mesh-network/mesh-gateway/mesh-gateway.ino)
  - Gateway sends heartbeat every 8 seconds, and all nodes on the network will receive it
  - If nodes don't receive it in time a new election will take place
    - Lowest node by ID will be elected and it will start sending heartbeats
  - Now we have to figure out how to send MAVLink data
    - So check MAVLink packet sizes and the packet sizes allowed from the painlessmesh library

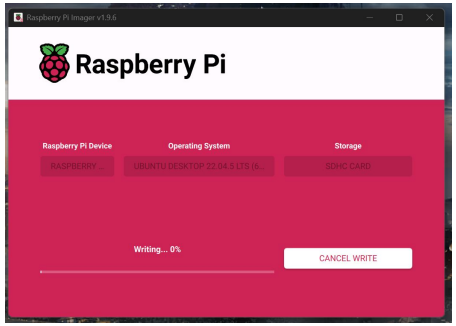
- Take a video for demonstration……

Mesh Multiple Connections Swarm Plans

- Each drone will have identicals setups
  - Each drone is hooked up with a Raspberry Pi capable of 5G, WiFi, Radio, and Bluetooth connections
  - Ground control connects to the drones by connecting to the raspberry pi via one of those connections
  - Once thats done we figure out how to handle connection switching among swarm and all that
- WiFi thoughts
  - It looks like mavesp8266 (which well need for esp mavlink stuff) doesnt support mesh networks and the painlessmesh stuff doesnt support mav connectivity and stuff so maybe we might need an extra esp for each drone? no thats too cumbersome... **perhaps we write the mavlink connectivity in the uploaded mesh stuff and forward it to the connected raspberry pi's mavproxy mav whatever stuff? might be the move…**
  - Otherwise we'd have to double up on esps (one for mavesp another for painlessmesh) and eventually double up for other communication channels (5G, radio, bluetooth)

Raspberry Pi for Pixhawk Connection (Method 1)

- Read the MicroSD with a MicroSD reader (or slot if you have)
  - Flash it with an OS
    - Raspberry PI 4 (unless you have something else)
    - I chose Ubuntu 22.04
      - “Other General Purpose OS” > “Ubuntu” > “Ubuntu Desktop 22.04”
    -
- Resources:
  - https://www.youtube.com/watch?v=kB9YyG2V-nA (best)
  - https://docs.px4.io/main/en/companion_computer/pixhawk_rpi
  - https://ubuntu.com/tutorials/how-to-install-ubuntu-desktop-on-raspberry-pi-4#1-overview



Raspberry Pi for Pixhawk Connection

Steps:

- Sudo raspi-config -> interface options -> login shell over serial No -> enable serial hardware Yes -> reboot
- Install mavlink-router -> git clone https://github.com/mavlink/mavlink-router.git -> cd mavlink-router -> sudo apt install -y ninja-build -> make - > sudo make install
- sudo mkdir -p /etc/mavlink-router -> sudo nano /etc/mavlink-router/main.conf

Note - the device could be different in the [UartEndpoint pixhawk]

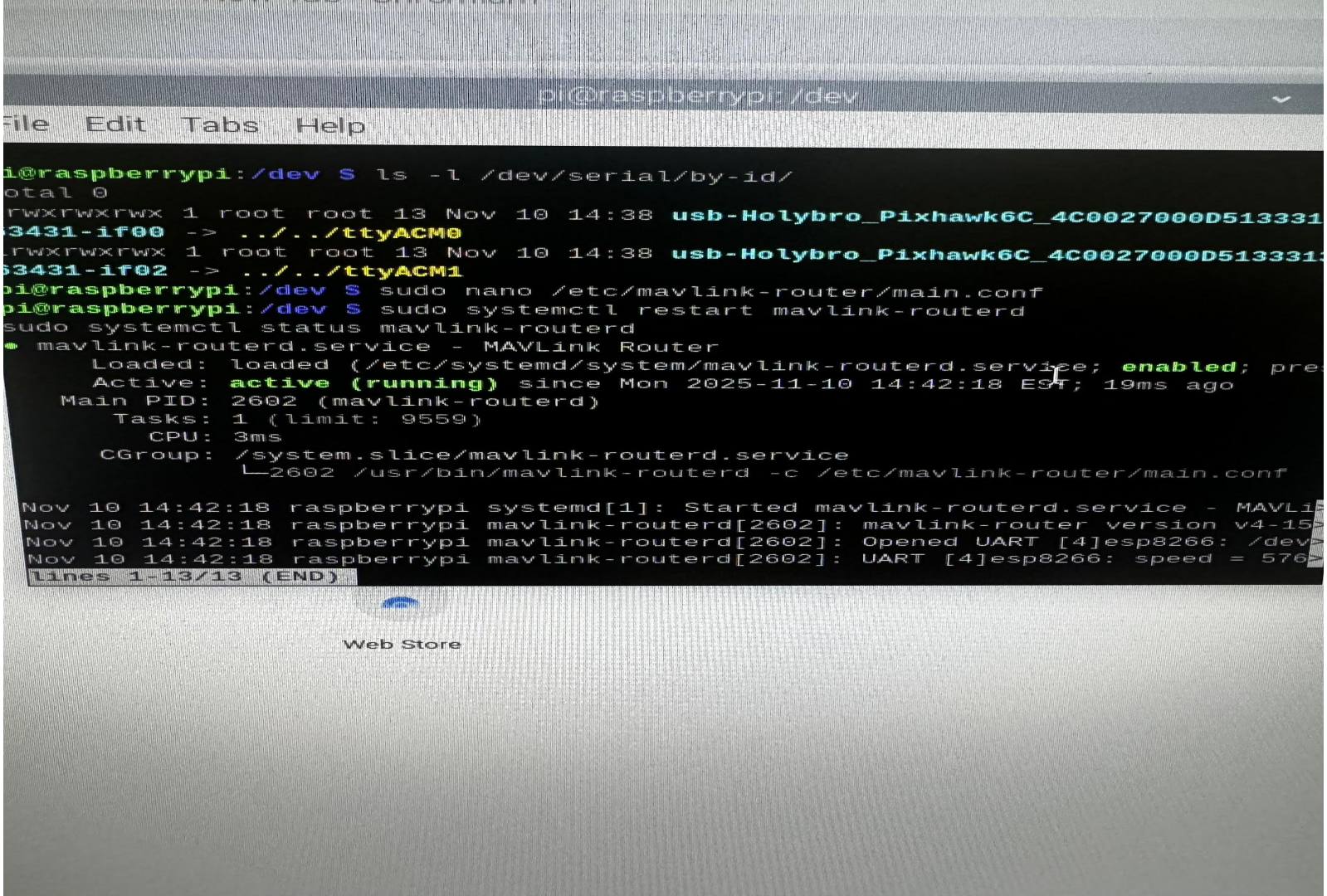
```
[General]
TcpServerPort=5760
UdpPort=14550
ReportStats=false

[UartEndpoint pixhawk]
Device=/dev/serial/by-id/usb-Holybro_Pix
hawk6C_4C0027000D51333130363431-i
f00
Baud=115200
FlowControl=false

[UartEndpoint esp8266]
Device=/dev/ttyAMA0
Baud=57600
FlowControl=false
```

Raspberry Pi for Pixhawk Connection Continued

- sudo nano /etc/systemd/system/mavlink-routerd.service Paste —>
- sudo systemctl daemon-reload
- sudo systemctl enable --now mavlink-routerd
- sudo systemctl status mavlink-routerd

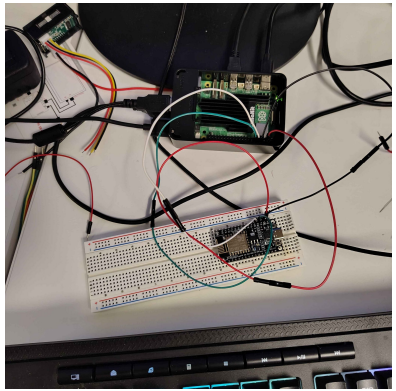
See next slide for expected output

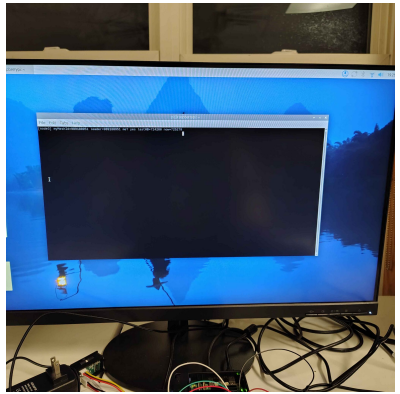
```
[Unit]
Description=MAVLink Router
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/mavlink-routerd -c /etc/mavlink-router/main.conf
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
```


pi@raspberrypi: /dev
File Edit Tabs Help
i@raspberrypi:/dev $ ls -l /dev/serial/by-id/
otal 0
rwxrwxrwx 1 root root 13 Nov 10 14:38 usb-Holybro_Pixhawk6C_4C0027000D513331
3431-if00 -> ../../ttyACM0
rwxrwxrwx 1 root root 13 Nov 10 14:38 usb-Holybro_Pixhawk6C_4C0027000D513331
3431-if02 -> ../../ttyACM1
pi@raspberrypi:/dev $ sudo nano /etc/mavlink-router/main.conf
pi@raspberrypi:/dev $ sudo systemctl restart mavlink-routerd
sudo systemctl status mavlink-routerd
mavlink-routerd.service - MAVLink Router
Loaded: loaded (/etc/systemd/system/mavlink-routerd.service; enabled; pre
Active: active (running) since Mon 2025-11-10 14:42:18 EST; 19ms ago
Main PID: 2602 (mavlink-routerd)
Tasks: 1 (limit: 9559)
CPU: 3ms
CGroup: /system.slice/mavlink-routerd.service
└2602 /usr/bin/mavlink-routerd -c /etc/mavlink-router/main.conf
Nov 10 14:42:18 raspberrypi systemd[1]: Started mavlink-routerd.service - MAVLi
Nov 10 14:42:18 raspberrypi mavlink-routerd[2602]: mavlink-router version v4-15
Nov 10 14:42:18 raspberrypi mavlink-routerd[2602]: Opened UART [4]esp8266: /dev
Nov 10 14:42:18 raspberrypi mavlink-routerd[2602]: UART [4]esp8266: speed = 576
Lines 1-13/13 (END)
Web Store

Pi - ESP Connection

- Ensure the drone and PI are powered down.
- Wiring:
  - Refer to image on the right
  - Connect ESP8266 GND to PI GND
  - Connect ESP8266 TX to PI RX
  - Connect ESP8266 RX TO PI TX
  - Connect ESP8266 3V3 to PI 3V3
- Boot the junk
- In terminal
  - Sudo apt install screen (so we can see whats going on)
  - Screen /dev/ttyAMA0 115200 (may need to adjust baud rate)
  - Should show the heartbeat logs
- Resources
  - https://www.instructables.com/Connect-an-ESP8266-to-your-RaspberryPi/
  -





Using mav-router (for esp testing only)

- Go into the main conf and comment out the section for pixhawk (since we only have one usb-c capable of data transfer)
- Check the status of mav-router (make sure there is no errors, should be exposing tcp)
- Remove 3V3 cable from breadboard (esp side), dont want conflicting power sources. Plug usb-c into computer and esp and open arduino ide and watch the serial monitor
- On the pi -> python3 -m venv ~/mavenv -> source ~/mavenv/bin/activate -> pip install --upgrade pip wheel setuptools -> pip install mavproxy future pyserial pymavlink numpy (dont need to do this unless your on a new pi) -> mavproxy.py --version
- Run this mavproxy.py --master=tcp:127.0.0.1:5760 --console and you should see on the esp serial monitor that the esp is receiving bytes (see next slide)

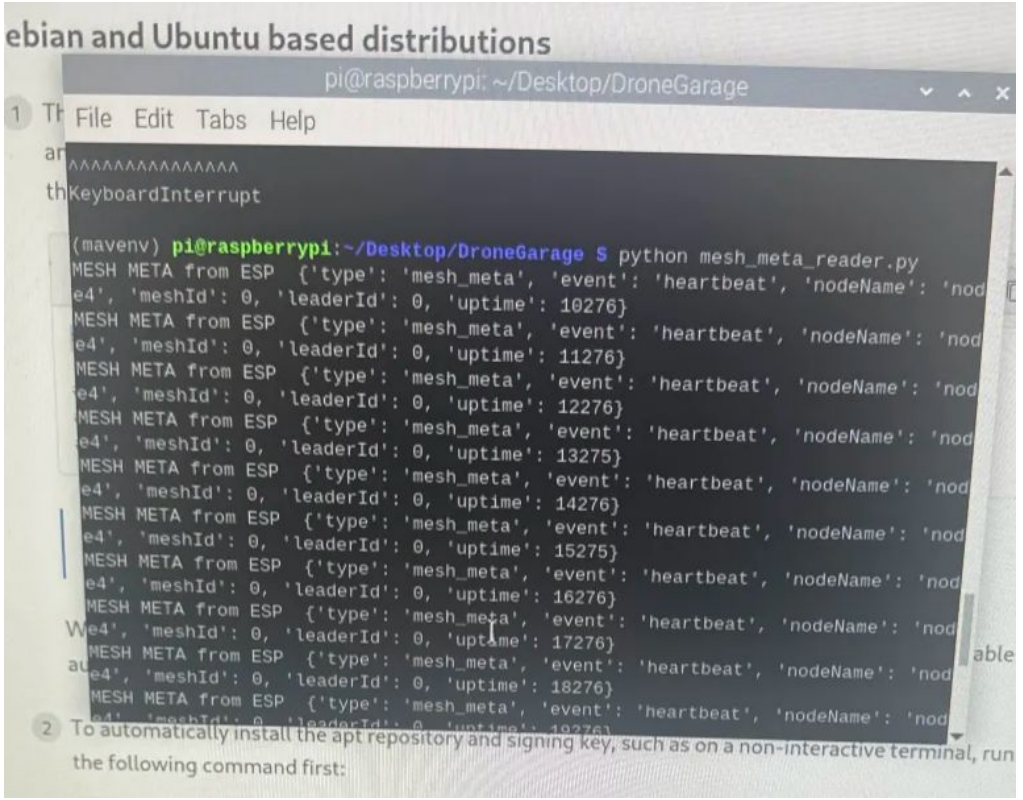
Message (Enter to send message to 'Generic ESP8266 Module' on 'COM6')

[tx] UART->mesh sent 29 bytes
[tx] UART->mesh sent 30 bytes
[node4] meshId=3167439714 leader=3167439714 me? yes lastHB=2131276 now=2132274
[tx] UART->mesh sent 11 bytes
[tx] UART->mesh sent 10 bytes
[node4] meshId=3167439714 leader=3167439714 me? yes lastHB=2132276 now=2133274
[tx] UART->mesh sent 21 bytes
[tx] UART->mesh sent 42 bytes
[tx] UART->mesh sent 13 bytes
[node4] meshId=3167439714 leader=3167439714 me? yes lastHB=2133276 now=2134274
[tx] UART->mesh sent 21 bytes
[node4] meshId=3167439714 leader=3167439714 me? yes lastHB=2134276 now=2135274
[tx] UART->mesh sent 21 bytes
[tx] UART->mesh sent 24 bytes
[tx] UART->mesh sent 31 bytes

Sending Info from ESP -> Pi

So since the Pi will be handing connections from the base station, it will need to know if its leader. From esp we send serial messages to the pi (heartbeats) letting it know that it is leader. And of course these messages only come in when that esp is leader. The esp also sends and event alert for if it has been elected leader or unelected leader

This is handled on the Pi through a python script listening for serial messages at baud rate 57600. The messages are tagged with @@ for easy decoding, and come in a json format



Flow of Running everything

Pi side:

Cd Desktop/DroneGarage

source ~/mavenv/bin/activate

sudo systemctl restart mavlink-routerd

sudo systemctl status mavlink-routerd

Python mesh_meta_reader.py

For ESP all you have to do is have it wired correctly and plugged in.

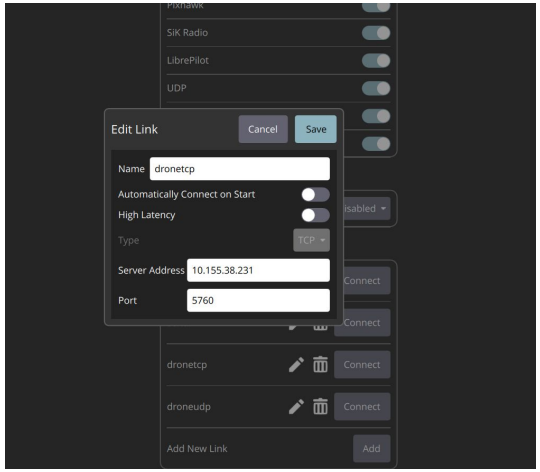
If the code isn't on the esp upload it through arduino ide

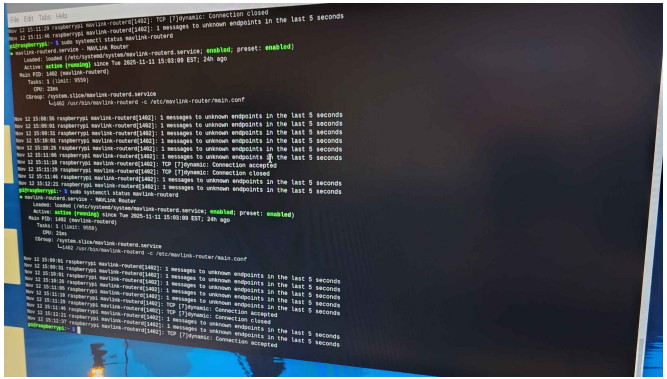
Forwarding QGC From ESP to Flight Controller via PI

- Problems as of writing
  - Steps to reproduce: Boot PI (with ESP connected) > Power on drone w/ battery > enable the mavlink-routerd system daemon stuff (refer to commands in previous slides) > running into problem where the tcp port is opened and not the udp?
    - Now that i think of it i think it opens a port on the pi and doesnt actually receive anything from the esp so maybe i have to upload code that forwards it?
  - A thought: if u cant get it to work using the esp8266WiFi or WifiUdp stuff or whatever... maybe further tinkering with the mesh network code will get desired results for forwarding? maybe theres a way for painlessmesh to write packets or data or whatever it received serially? in that way it writes it to the serial connection with the pi which the router will see and route to the serial connection with the pixhawk?
- Resources (& Libraries)
  - <ESP8266WiFi.h>
  - <WiFiUdp.h>
  - https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/udp-examples.html

QGC to PI to Drone (Firstly)

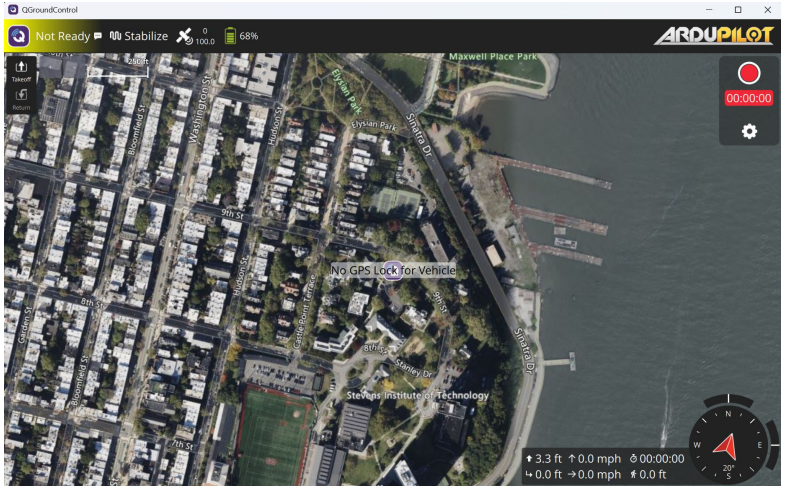
- First try: edit main.conf
  - Commenting out esp [endpoint]
  - Keeping pixhawk [endpoint]
  - ~~Adding tcp or udp [endpoint] (whichever works)~~
    - Wait maybe??? Idk hold on
  - >Ip address to get ip
  - Open up QGC → Application Settings → Comm Links → Set up TCP connection using the IP from “Ip address” command and port in main.conf (usually 5760) → Connect → Works! (ish)
    - Refer to images on the right
    - Maybe it works? But I still cant see vehicle configuration data… maybe i have to send something?
- References
  - https://github.com/mavlink-router/mavlink-router/blob/master/examples/config.sample
    - Main.conf example

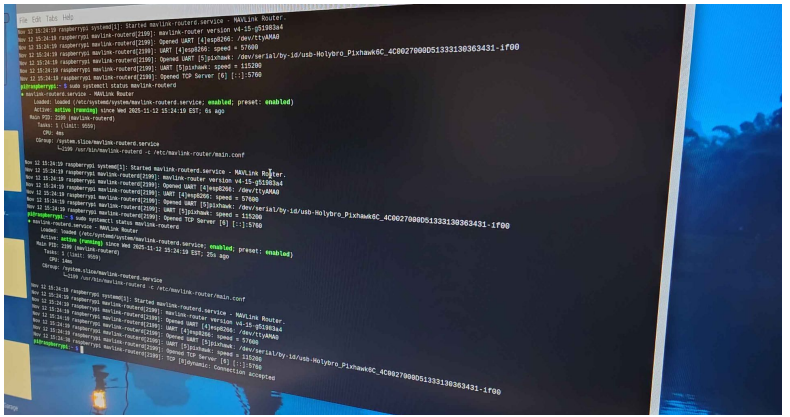




QGC to PI to Drone

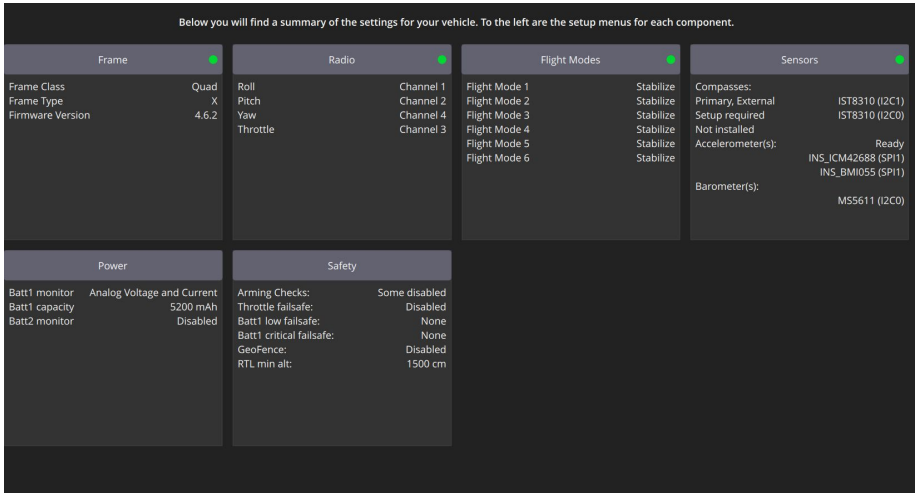
- As for what to test the connection with? Idk… here's a screenshot
- **Just make sure that u run "sudo systemctl restart mavlink-routerd" on the pi → "sudo systemctl status mavlink-routerd" to ensure that the serial and tcp endpoints are opened up → grab ip address of pi and tcp port in main.conf and use that for a tcp connection in QGC's Application Settings/Comm Links section**
- Now on to handling elections with the mavlink router





Problems? Maybe?

- Now its connected via TCP and stuff you'd expect it to show up on the "Vehicle Configuration" page right? Not quite (refer to image)
- Perhaps this is because of the connection being to the pi itself as opposed to the drone (again, the pi routes the connection from its tcp endpoint to the serially connected drone and vice versa)
- I mean i guess this is fine?



Elections & IP Change Challenges (Minor-ish)

- Goal here is that when a drone fails or disconnects or smth a new drone is elected for leadership, right? And now we want it so that when that drone fails, its IP is now the newly elected drone's IP, right? Say we want to send a signal to the pi that it's the new leader, great for single drone case but what about for multiple drones? Assuming the drone that dc'd is a leader and joins back, its pin that signals to its pi for leadership will be set to HIGH (it doesnt toggle back to its default state on disconnect it must be toggled while running), meaning that its pi will do the thing that gets the gateway IP when it's not the gateway drone and potentially messing w the QGC connection…
  - Solutions?
    - Maybe set up a thing where the drone that joins first checks if theres already a leader by setting its pin to LOW… nah the issue is that the pi on boot will automatically look at that pin and if its HIGH that means itll instantly do the thing… **maybe we have the pi toggle it back to LOW once it does its thing??? LOWKEY**
      - **Nah maybe we'll have to have a second gpio connection instead**
    - Maybe theres a thing where u can send a pulse of sorts instead of a toggle?

Elections & IP Change - Set Up and Further Thoughts Ig

- Note the GPIO numbers being used in both the pi and the esp btw
  - For myself, using GPIO5 (D1) on the esp with GPIO18 on the pi ~~and GPIO4 (D2) on the esp with GPIO17 on the Pi~~
    - Refer to image on the right
    - Another note, maybe broadcast the IP as well bc we dont know when the leader wil lgo down?
    - Wait hold on in the pi wouldnt it be that the pi will always set it off
  - Hold on maybe have the pi detect a toggle instead of two pins?
    - Maybe we send out a LOW once from the pi to the esp and just straight toggle from the esp to the pi?
  - Or maybe if the esp is a leader is just keeps toggles?
  - **Or maybe we have some blocking functionality? The pi first blocks until the gpio is toggled by the esp then it takes the mesh's ip address**
    - Only way i can think of it working without the pi on boot reading a signal and immediately yoinking that ip thinking its the leader
- Resources
  - Wiring stuff
    - https://randomnerdtutorials.com/esp8266-pinout-reference-gpios/
    - https://hackatronic.com/raspberry-pi-5-pinout-specifications-pricing-a-complete-guide/

UART Drone Communication (Reading MAVLink on ESP)

- Libraries?
  - #include <MAVLink.h>
    - Obtain in Arduino IDE:
      - Sketch > Include Library > Manage Libraries > Search “mav” > Select “MAVLink by Oleg Kalachev”
        - Theres two options i chose this bc the other was last updated 5~7 years ago (eek)
- Implementation
  -
- Resources
  - https://discuss.ardupilot.org/t/mavlink-and-arduino-step-by-step/25566
  - https://github.com/okalachev/mavlink-arduino
  - https://github.com/okalachev/mavlink-arduino/tree/master/mavlink/common
    - MAVLink.h function references

Possible Pi 5G Options

- Mad expensive (and out of stock i think)
  - https://sixfab.com/product/raspberry-pi-5g-development-kit-5g-hat/
    - Put the junk on a PI 4 and make it 5G ready
    - Works everywhere but China apparently (in the description verbatim)
    - Doesn't come with a PI
  - https://sixfab.com/product/sixfab-5g-modem-kit-for-raspberry-pi-5/
    - Put the junk on a PI 5 and make it 5G ready
    - Works everywhere globally or something like that
    - Doesn't come with a PI (I think)
  - https://sixfab.com/product/sixfab-jumpstart-5g/
    - Comes with a PI (in fact based on one so its like already set up)
    - Lowkey looks mid compared to the rest
- Other options
  - https://www.pishop.us/product/pcie-to-m-2-4g-5g-and-usb-3-2-hat-for-raspberry-pi-5/
    - PI 5 Hat for M.2 5G slot or something like that (allows for high performance as opposed to some usb connection thats the same price)
    - 5G module not included (options below)
      - https://www.waveshare.com/rm530n-gl.htm?utm_source=chatgpt.com
        - Mad expensive but really only thing i can find
      - https://www.waveshare.com/rm520n-gl.htm?utm_source=chatgpt.com
        - Cheaper – still expensive though
    - Antennas
      - https://www.modalai.com/products/mrp-d0004-1-06
        - Absolutely non budget option
      - https://connecteninternet.com/products/replacement-antenna-4g-5g
        - Maybe this??? Doesn't look well-documented so idk
    - NanoSIM
      - https://www.walmart.com/ip/Verizon-5G-SA-NANO-SIM-4G-5G-LTE-Network-Brand-New/1965007443?wmlspartner=wlpa&selectedSellerId=6134
      - Definitely other options (easiest thing here to procure)

Ishan’s Recommendations

- Yeah the Quectel modems are out of stock everywhere except waveshare. You can order the modem from there and the hat from 6 fab.
- Hat without the modem: https://sixfab.com/product/raspberry-pi-5g-development-kit-5g-hat/
- Modem: https://www.waveshare.com/rm530n-gl.htm?utm_source=chatgpt.com

Next Steps

- When the esp network elects a new leader the corresponding pi will become the active gateway
  - A script will monitor the esp for election messages, and if elected it will start it router with TCP port open for QGC. If it's no longer leader it will stop or close it TCP listener

- QGC will connect to the current gateway PI using a fixed IP
  - So the new leader will claim that IP via a Python script, so QGC can automatically reconnect

- Test connections like:
  - QGC -> Gateway Pi -> ESP mesh -> Drone Pi’s -> Pixhawk
  - Pixhawk -> Drone Pi -> ESP mesh -> Gateway Pi -> QGC